

Infrastructure as Code Deployment and Automation using Terraform

Project Name: CAREERHUB

Submitted By: Saniya Maulavi

Course: DevOps with AWS

Guided By: Akhilesh Dubey

Date: 02/10/2026

Introduction

CareerHub is a web-based career and job application platform developed as part of this project. The main objective of this project was to deploy and manage the infrastructure required for CareerHub using Infrastructure as Code (IaC).

Instead of manually creating and configuring every AWS resource, Terraform was used to define the infrastructure through configuration files. This makes the infrastructure repeatable, organized and easier to manage.

The project was performed using an AWS EC2 instance as the Terraform control machine. Terraform and AWS CLI were installed and configured on this machine. Terraform was then used to create the AWS infrastructure required for CareerHub.

The project also included practical implementation of Terraform modules, variables, outputs, infrastructure lifecycle, remote state, state management and infrastructure drift detection.

As additional Infrastructure as Code technologies, AWS CloudFormation and AWS CDK were also implemented for CareerHub. Git and GitHub were used for version control and to maintain the complete project repository.

Project Objectives

The main objectives of the CareerHub project were:

1. To understand the concept of Infrastructure as Code.
2. To install and configure Terraform on an AWS EC2 instance.
3. To configure and verify AWS CLI.
4. To provision AWS infrastructure using Terraform.
5. To create a VPC and subnet using Terraform.
6. To configure an Internet Gateway and route table.
7. To create and configure a Security Group.
8. To provision an EC2 instance using Terraform.
9. To create and associate an Elastic IP.
10. To create an Amazon S3 bucket.
11. To understand Terraform variables and outputs.
12. To organize infrastructure using reusable Terraform modules.
13. To understand Terraform infrastructure lifecycle.
14. To configure Terraform remote state using Amazon S3.
15. To demonstrate Terraform state management.
16. To demonstrate infrastructure drift detection.
17. To deploy the CareerHub application on the Terraform-provisioned EC2 instance.
18. To implement similar infrastructure using AWS CloudFormation.
19. To implement similar infrastructure using AWS CDK.
20. To maintain the project using Git and GitHub.

Project Environment and AWS Setup

The project was implemented using Amazon Web Services. The AWS region selected for the project was **ap-southeast-2 (Sydney)**.

A dedicated EC2 instance named **CareerHub-Terraform-Control** was used as the Terraform control machine. Terraform commands, AWS CLI commands, CloudFormation commands and AWS CDK commands were executed from this environment.

Terraform version **1.16.4** was installed and verified successfully.

AWS CLI was also installed and configured. The configured AWS CLI version was:

`aws-cli/2.31.35`

The AWS identity was verified using AWS STS to make sure that the configured AWS credentials were working correctly.

A project directory named:

`careerhub-tf`

was created on the control EC2 instance.

The AWS environment was therefore prepared before beginning the infrastructure provisioning tasks.

The project used the **c7i-flex.large** instance type for the CareerHub infrastructure and a **30 GB encrypted gp3** root volume.

Terraform Configuration

Terraform was used as the main Infrastructure as Code tool for the CareerHub infrastructure.

The Terraform configuration was divided into multiple files so that different parts of the configuration could be managed separately.

The main Terraform files were:

- provider.tf
- main.tf
- variables.tf
- outputs.tf

The provider.tf file contains the Terraform provider configuration and the S3 backend configuration for remote state.

The main.tf file calls the reusable Terraform modules.

The variables.tf file contains input variables such as:

- AWS region
- VPC CIDR
- Subnet CIDR
- Availability Zone
- AMI ID
- Instance type
- Key pair name
- Root volume size
- S3 bucket prefix

The outputs.tf file provides important information after deployment, including VPC ID, subnet ID, security group ID, EC2 instance ID, public IP, Elastic IP and S3 bucket name.

Terraform initialization was performed successfully and the AWS provider was installed.

AWS Infrastructure Provisioning

Terraform was used to provision the main AWS infrastructure required for CareerHub.

The infrastructure included:

- VPC
- Subnet
- Internet Gateway
- Route Table
- Route Table Association
- Security Group
- EC2 Instance
- Elastic IP
- Elastic IP Association
- S3 Bucket

The VPC was created with the CIDR block:

10.0.0.0/16

The subnet was created with:

10.0.1.0/24

The subnet was configured as a public subnet so that the EC2 instance could communicate through the Internet Gateway.

A route table was configured with a default route:

0.0.0.0/0

pointing towards the Internet Gateway.

Terraform plan was used before applying the infrastructure. The plan showed the resources that Terraform intended to create.

Terraform apply was then executed successfully and the AWS infrastructure was provisioned

Security Group and EC2 Configuration

A dedicated Security Group was created for the CareerHub infrastructure.

The Security Group allowed the following inbound ports:

- Port 22 – SSH
- Port 80 – HTTP
- Port 443 – HTTPS
- Port 5173 – CareerHub application

The Security Group allowed outbound traffic so that the EC2 instance could communicate with required external services.

The Terraform-managed EC2 instance was configured with:

Instance Type: c7i-flex.large

Root Volume: 30 GB gp3

Volume Encryption: Enabled

Key Pair: CareerHub-Terraform

Operating System: Ubuntu

An Elastic IP was also created and associated with the Terraform-managed EC2 instance.

The Elastic IP provided a stable public address for accessing the CareerHub application.

The final Terraform deployment successfully created the required networking, security and compute resources.

Terraform Modules

The Terraform infrastructure was organized into reusable modules.

The project contains four main modules:

VPC Module

The VPC module creates:

- VPC
- Subnet
- Internet Gateway
- Route Table
- Route Table Association

Security Group Module

The Security Group module creates the CareerHub Security Group and defines the required inbound and outbound rules.

EC2 Module

The EC2 module creates:

- EC2 instance
- Elastic IP
- Elastic IP association

S3 Module

The S3 module creates the CareerHub S3 bucket.

The root main.tf file calls these modules and passes the required variables to them.

Using modules makes the infrastructure configuration more organized and reusable. Instead of keeping every AWS resource in one large file, related resources are grouped into separate modules.

Terraform formatting and validation were successfully performed after creating the modules.

Terraform Variables, Outputs and Lifecycle

Terraform variables were used to make the infrastructure configuration flexible.

Instead of directly repeating values throughout the configuration, important values were defined as variables.

Examples include:

- AWS region
- VPC CIDR
- Subnet CIDR
- Availability Zone
- AMI ID
- Instance type
- Key pair
- Root volume size

Terraform outputs were used to display important information after infrastructure deployment.

The outputs included:

- VPC ID
- Subnet ID
- Security Group ID
- EC2 Instance ID
- EC2 Public IP
- Elastic IP
- S3 Bucket Name

The Terraform lifecycle was also demonstrated.

Terraform was used to:

- Create infrastructure
- Modify infrastructure
- Apply updates
- Destroy infrastructure when required

terraform plan was used to review changes before applying them.

terraform apply was used to apply the changes.

terraform destroy was used during the project lifecycle to remove an earlier Terraform deployment when required for the module-based implementation.

CareerHub Website Deployment

After the Terraform infrastructure was successfully provisioned, the CareerHub application was deployed on the Terraform-created EC2 instance.

The CareerHub application source code was stored in GitHub.

The repository was cloned onto the Terraform worker EC2 instance.

Node.js and npm were installed on the worker instance.

The project dependencies were installed using:

```
npm ci
```

The CareerHub application uses Vite, so the application was started using:

```
npm run dev -- --host 0.0.0.0
```

The application was running on port **5173**.

Terraform was then used to allow TCP port 5173 through the CareerHub Security Group.

After applying the Security Group change, the CareerHub application was accessed through the Elastic IP of the Terraform-provisioned EC2 instance.

The application was successfully displayed in the browser, confirming that CareerHub was deployed on the Terraform-managed AWS infrastructure.

Terraform Remote State

Terraform remote state was configured using Amazon S3.

A dedicated S3 bucket was created for storing the Terraform state:

`careerhub-terraform-state-1790939597`

The bucket was created in the AWS Sydney region.

The Terraform backend was configured to use Amazon S3.

The state file was stored using the key:

`careerhub/terraform.tfstate`

Encryption was enabled for the backend.

S3 versioning was also enabled for the remote state bucket.

The local Terraform state was migrated to the S3 backend using:

`terraform init -migrate-state`

Terraform successfully configured the S3 backend and migrated the existing state.

The remote state file was verified using the AWS CLI.

The state object was found inside the expected S3 location.

Terraform state files were not committed to GitHub.

Remote state provides centralized storage of Terraform state and makes state management more suitable for Infrastructure as Code projects.

Terraform State Management

Terraform state maintains information about the resources managed by Terraform.

The project demonstrated basic Terraform state management using different Terraform commands.

The command:

```
terraform show
```

was used to display detailed information about the current Terraform state.

The command:

```
terraform state list
```

was used to display all resources currently managed by Terraform.

The command:

```
terraform state show
```

was used to inspect an individual resource.

For example, the Terraform-managed VPC was inspected using the state show command.

Finally, terraform plan was executed to compare the Terraform configuration with the actual AWS infrastructure.

The result was:

No changes. Your infrastructure matches the configuration.

This confirmed that the actual infrastructure was synchronized with the Terraform configuration.

State management helped verify which AWS resources were currently under Terraform control.

Infrastructure Drift Detection

Infrastructure drift happens when the actual AWS infrastructure is changed outside Terraform.

A drift demonstration was performed on the CareerHub Security Group.

First, Terraform was checked using:

```
terraform plan
```

The infrastructure was initially synchronized with the Terraform configuration.

After that, the Security Group Name tag was manually changed using AWS CLI.

The manual change was performed outside Terraform.

Terraform was then executed again using:

```
terraform plan
```

Terraform detected the difference between the actual AWS Security Group and the expected configuration.

The plan showed that the manually changed Name tag would be changed back to the value defined in Terraform.

The configuration was restored using:

```
terraform apply
```

After applying the correction, terraform plan was executed again.

The final result showed no changes.

This demonstrated that Terraform can identify infrastructure changes made outside the Terraform configuration and can restore the desired configuration.

AWS CloudFormation Implementation

AWS CloudFormation was implemented as another Infrastructure as Code approach for CareerHub.

A CloudFormation template named:

careerhub-stack.yaml

was created.

The template included the following AWS resources:

- VPC
- Subnet
- Internet Gateway
- Route Table
- Route Table Association
- Security Group
- EC2 Instance
- Elastic IP
- Elastic IP Association
- S3 Bucket

Parameters were added for values such as VPC CIDR, subnet CIDR, Availability Zone, instance type, key pair and root volume size.

The template was first validated using the AWS CloudFormation validation command.

After successful validation, the CloudFormation stack:

CareerHub-CloudFormation-Stack

was deployed successfully.

The stack outputs were checked using the AWS CLI.

The deployment successfully created the required CareerHub CloudFormation infrastructure in the AWS Sydney region.

AWS CDK Implementation

AWS CDK was also implemented for CareerHub.

A basic AWS CDK project was created using **TypeScript**.

Node.js and npm were installed on the control EC2 and the AWS CDK CLI was installed.

The CDK project was initialized using the TypeScript template.

The CDK stack creates:

- VPC
- Public Subnet
- Security Group
- EC2 Instance
- Elastic IP
- S3 Bucket

The EC2 instance was configured with the c7i-flex.large instance type, CareerHub-Terraform key pair and a 30 GB encrypted gp3 volume.

The CDK project was compiled using:

```
npx tsc --noEmit
```

The infrastructure template was generated using:

```
npx cdk synth
```

The AWS CDK environment was bootstrapped before deployment.

During the first deployment, an Elastic IP association issue was identified. The configuration was corrected to use the Elastic IP allocation ID.

After the correction, the project was compiled and deployed successfully.

The final CDK deployment produced the required AWS resources and outputs.

Terraform, CloudFormation and AWS CDK Comparison

Terraform, AWS CloudFormation and AWS CDK are all Infrastructure as Code technologies, but they use different approaches.

Feature	Terraform	CloudFormation	AWS CDK
Developed By	HashiCorp	AWS	AWS
Configuration	HCL	YAML / JSON	TypeScript and other languages
Main Focus	Multi-cloud IaC	AWS IaC	AWS IaC using programming languages
State	Terraform State	CloudFormation Stack	CloudFormation
Reusability	Modules	Templates / Nested Stacks	Constructs
Deployment	terraform plan / apply	Stack deployment	cdk synth / deploy
Cloud Support	Multi-cloud	AWS	AWS
Project Usage	Main implementation	Alternative implementation	Alternative implementation

Terraform was used as the main infrastructure tool in this project.

CloudFormation demonstrated the AWS-native Infrastructure as Code approach.

AWS CDK demonstrated how infrastructure can be defined using a programming language such as TypeScript.

Implementing all three approaches provided practical understanding of their configuration and deployment workflows.

Git and GitHub

Git and GitHub were used to maintain the CareerHub Infrastructure as Code project.

A separate GitHub repository was created for the infrastructure project:

CareerHub-Terraform-Infrastructure

The existing CareerHub application repository was kept separate from the infrastructure repository.

Git was initialized inside the Terraform project directory.

A .gitignore file was created to prevent unnecessary or sensitive files from being committed.

The .gitignore includes exclusions for:

- Terraform state files
- Terraform local files
- .tfvars files
- PEM keys
- Environment files
- CDK generated files
- Node modules
- Editor files

The project was committed to Git and pushed to GitHub.

The repository contains the Terraform configuration, modules, CloudFormation template, CDK project, README documentation and project screenshots.

No AWS secret keys or Terraform state files were uploaded to GitHub.

Challenges Faced and Solutions

Several practical challenges were encountered during the implementation.

CareerHub Application Access

The CareerHub Vite application required port 5173 to be accessible externally. The Security Group was updated through Terraform to allow TCP port 5173.

Terraform Remote State

The local Terraform state had to be migrated to Amazon S3. The S3 backend was configured and terraform init -migrate-state was used to migrate the existing state.

Infrastructure Drift

A manual Security Group change was intentionally introduced to demonstrate drift. Terraform plan detected the difference and Terraform apply restored the expected configuration.

AWS CDK Deployment

The first CDK deployment encountered an Elastic IP association issue because the IP reference was used instead of the required allocation ID. The CDK code was corrected and the deployment succeeded.

Git Repository Structure

The CDK initialization created Git metadata inside the CDK directory. The nested Git metadata was removed so that the complete project could be maintained as one Git repository.

These challenges helped provide practical understanding of troubleshooting and Infrastructure as Code workflows.

Conclusion

The CareerHub project successfully demonstrated Infrastructure as Code using Terraform on AWS.

Terraform was used to provision the required AWS infrastructure including networking, security, EC2, Elastic IP and S3 resources. The infrastructure was organized using reusable modules and configured using variables and outputs.

The CareerHub application was successfully deployed on the Terraform-provisioned EC2 instance and verified through its public Elastic IP.

Terraform state management was demonstrated using state inspection commands, and remote state was configured using Amazon S3. Infrastructure drift was also demonstrated by making a manual AWS change and allowing Terraform to detect and correct it.

AWS CloudFormation and AWS CDK were implemented to understand alternative Infrastructure as Code approaches.

Git and GitHub were used for version control and project management. The final repository contains the infrastructure configuration, modules, CloudFormation template, CDK project, documentation and 39 screenshots.

Overall, the project provided practical experience in AWS infrastructure provisioning, Terraform, Infrastructure as Code, application deployment, state management, drift detection, CloudFormation, AWS CDK and Git/GitHub.